

DAY 4 - PostgreSQL 18 Backup & Disaster Recovery

LIVE Patroni / Zalando Postgres Operator Cluster - SINGLE-LINE COMMAND EDITION

Environment: namespace **postgres-ha** | cluster **acid-postgres-ha** | Docker Kubernetes | GCS bucket **postgres-ha-backups-parameswari-2026**

COPY/PASTE RULE: Every shell command below is one physical command line. Configuration files are shown separately as file contents and are not shell commands.

PostgreSQL 18 Backup & Disaster Recovery

Step-by-Step Lab Manual for a LIVE Patroni / Zalando Postgres Operator Cluster Namespace postgres-ha Cluster acid-postgres-ha (Zalando Operator + Patroni) PostgreSQL 18 Backup tools pg_dump, pg_dumpall, pg_basebackup, WAL archiving, pgBackRest Enterprise backup Full / Incremental / Differential / Compression / Encryption / Retention / PITR Restore rule Restore only to isolated recovery storage / separate recovery cluster CRITICAL SAFETY RULE: Take backups from the live cluster, but never overwrite the active Patroni member PVCs during restore or PITR. Perform physical restore only on isolated recovery storage or a separate recovery cluster. COPY/PASTE RULE: Every shell command in this corrected edition is a separate paragraph. Run commands one at a time. Do not paste two commands onto the same shell line unless they are intentionally joined with &&.

Table of Contents

- 0. Lab Architecture, Scope and Safety
- 1. Pre-Lab Validation of the Live Patroni Cluster
- 2. Backup Concepts: Logical, Physical, WAL and PITR
- 3. Lab 1 - Logical Backup from the Live Cluster
- 4. Lab 2 - Physical Backup with pg_basebackup (Corrected)
- 5. Lab 3 - WAL Archiving Validation
- 6. Lab 4 - Configure pgBackRest for the Live Cluster
- 7. Lab 5 - Full Backup with pgBackRest
- 8. Lab 6 - Incremental Backup
- 9. Lab 7 - Differential Backup
- 10. Repository, Compression, Encryption and Retention
- 11. Lab 8 - Restore / Full Cluster Recovery
- 12. Lab 9 - Point-in-Time Recovery (PITR)
- 13. Patroni Failover Considerations During Backup
- 14. Validation, Monitoring, Troubleshooting and Cleanup
- 15. RPO / RTO and Suggested Backup Schedule
- 16. Final Coverage Checklist

Implementation note: Zalando/Spilo commonly uses WAL-G/WAL-E for native backup. pgBackRest is not automatically present in all Spilo images. The pgBackRest portion therefore assumes an immutable image that contains a PostgreSQL 18-compatible pgBackRest build and a durable repository. Do not install packages ad hoc into a running production pod and assume they will survive restart.

0. Lab Architecture, Scope and Safety

0.1 What runs live and what is isolated?

- LIVE: acid-postgres-ha remains Patroni-managed and continues serving traffic during backup exercises.
- LIVE BACKUP: logical dumps, pg_basebackup, WAL archive checks and pgBackRest backup commands

run against the current primary or an eligible replica.

- ISOLATED RESTORE: physical restore and PITR use separate recovery storage. Never replace files below active PGDATA while Patroni is running.

- REPOSITORY: use durable object storage (GCS/S3) or, for training only, an RWX volume reachable by every member.

0.2 Recovery architecture

Application -> acid-postgres-ha Service -> Patroni Primary
Primary -> logical backup / physical backup / WAL archive / pgBackRest repository
Repository + archived WAL -> isolated recovery PGDATA -> validation -> controlled cutover

0.3 Why failover is not backup

Patroni protects availability. It does not protect against logical mistakes such as DROP TABLE, accidental DELETE, bad application updates, corruption that propagates to replicas, or retention mistakes. Backup and PITR provide recovery from those events.

1. Pre-Lab Validation of the Live Patroni Cluster

1.1 Set namespace and inspect resources

kubectl config set-context --current --namespace=postgres-ha
kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc
kubectl get statefulset

1.2 Set variables

NS=postgres-ha
CLUSTER=acid-postgres-ha

1.3 Identify the current primary - do not assume pod-0

for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do echo "=== \$p ==="; kubectl -n "\$NS" exec "\$p" -- curl -s http://127.0.0.1:8008/ jq -r '"role=" + (.role // "unknown") + " state=" + (.state // "unknown")'; done

Exactly one member should report primary/leader. If the cluster is unhealthy, stop the backup lab and repair HA first.

1.4 Set PRIMARY and REPLICA after checking

PRIMARY=acid-postgres-ha-0
REPLICA=acid-postgres-ha-1

Change both values to match the actual current primary and a healthy replica.

1.5 Validate role, version, PGDATA and LSN

kubecttl -n "\$NS" exec "\$PRIMARY" -- psql -U postgres -Atqc "SELECT pg_is_in_recovery(), current_database(), version();"
kubecttl -n "\$NS" exec "\$PRIMARY" -- psql -U postgres -Atqc "SHOW data_directory;"
kubecttl -n "\$NS" exec "\$PRIMARY" -- psql -U postgres -c "SELECT now(), pg_current_wal_lsn(), pg_is_in_recovery();"
kubecttl -n "\$NS" exec "\$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr, state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn FROM pg_stat_replication;"

2. Backup Concepts: Logical, Physical, WAL and PITR

Logical backup (pg_dump/pg_dumpall) stores SQL objects and data and supports database/object-level restore. Physical backup (pg_basebackup/pgBackRest) copies the cluster data files and is the foundation for cluster-level recovery and PITR. WAL archiving continuously preserves transaction history. PITR restores a physical backup and replays archived WAL up to a selected time/LSN/transaction.

2.1 Full vs differential vs incremental

- Full: complete physical backup; establishes an independent backup baseline.
- Differential: changes since the most recent full backup. Restore uses that full + the chosen differential +

required WAL.

- Incremental: changes since the immediately preceding backup. Restore can require a chain of dependent

backups + required WAL.

- Archived WAL is not a substitute for a base backup; it is the continuous roll-forward stream used after

restoring a physical backup.

2.2 RPO and RTO

RPO is acceptable data loss. Frequent, healthy WAL archiving reduces potential data loss. RTO is acceptable service restoration time; it depends on backup size, storage/network speed, number of backup dependencies, WAL replay volume and validation time.

3. Lab 1 - Logical Backup from the Live Cluster

3.1 Create local backup directory

mkdir -p backups/logical

3.2 Full cluster logical backup

kubecttl -n "\$NS" exec "\$PRIMARY" -- pg_dumpall -U postgres --clean --if-exists > "backups/logical/\${CLUSTER}-dumpall-\$(date +%Y%m%d-%H%M%S).sql"
ls -lh backups/logical/

3.3 Custom-format backup of banking database

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dump -U postgres -Fc -d banking -f /tmp/banking.dump

kubectl -n "$NS" cp "$PRIMARY:/tmp/banking.dump" "backups/logical/banking-$(date +%Y%m%d-%H%M%S).dump"
```

3.4 Verify without restoring

```
BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)

echo "$BACKUP_FILE"

pg_restore -l "$BACKUP_FILE" | head -40

SQL_FILE=$(ls -lt backups/logical/${CLUSTER}-dumpall-*.sql | head -1)

head -30 "$SQL_FILE"
```

3.5 Test logical restore into an isolated test database

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "DROP DATABASE IF EXISTS banking_restore_test;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "CREATE DATABASE banking_restore_test;"

BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)

echo "$BACKUP_FILE"

kubectl -n "$NS" cp "$BACKUP_FILE" "$PRIMARY:/tmp/banking-restore.dump"

kubectl -n "$NS" exec "$PRIMARY" -- pg_restore -U postgres -d banking_restore_test --clean --if-exists /tmp/banking-restore.dump

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking_restore_test -c "\dt"
```

On a real production system, run restore tests in a dedicated recovery cluster instead of consuming I/O/storage on the production primary.

4. Lab 2 - Physical Backup with pg_basebackup (Corrected)

This section corrects the original manual. pg_basebackup uses the PostgreSQL replication protocol. A normal superuser login is not enough if pg_hba.conf does not allow that user for replication. In this Zalando/Patroni cluster, use the cluster primary Service and the operator-managed replication/standby credentials.

4.1 Verify the primary Service name

```
kubectl -n "$NS" get svc

PRIMARY_SERVICE=acid-postgres-ha

kubectl -n "$NS" exec "$REPLICA" -- getent hosts "$PRIMARY_SERVICE"
```

Do not use -h 127.0.0.1 when the client command is running in a replica pod. 127.0.0.1 means that same replica pod. Do not use -h postgres-ha unless that Service actually exists. For this cluster, the primary Service is acid-postgres-ha.

4.2 Inspect replication state and lag

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr, state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag FROM pg_stat_replication;"
```

4.3 Identify replication-capable roles

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT rolname, rolreplication, rolcanlogin FROM pg_roles WHERE rolreplication = true;"
```

4.4 Check replication HBA rules

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT line_number, type, database, user_name, address, auth_method, error FROM pg_hba_file_rules WHERE 'replication' = ANY(database);"
```

Do not add broad rules such as host replication all 0.0.0.0/0 trust. Use the operator-managed replication role and existing HBA policy.

4.5 Find the operator-managed replication/standby secret

```
kubectl -n "$NS" get secrets | grep -Ei "standby|replic"
```

If the cluster exposes the standard Zalando standby credential secret, set it as follows. If the actual secret name differs, substitute the name returned by the previous command.

```
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
```

4.6 Load replication username and password into your local shell

```
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.username}' | base64 -d)
```

```
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.password}' | base64 -d)
```

```
echo "$REPL_USER"
```

Do not echo the password during training or production. Keep it only in the shell variable and unset it after the exercise.

4.7 Test ordinary connectivity with the replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" psql -h "$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" -d postgres -c "SELECT pg_is_in_recovery(), inet_server_addr(), inet_server_port();"
```

Expected: pg_is_in_recovery() = false because acid-postgres-ha routes to the current primary.

4.8 Prepare backup directory in the client pod

```
kubectl -n "$NS" exec "$REPLICA" -- rm -rf /tmp/basebackup
```

```
kubectl -n "$NS" exec "$REPLICA" -- mkdir -p /tmp/basebackup
```

4.9 Run pg_basebackup with the correct host and replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" pg_basebackup -h "$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" -D /tmp/basebackup -Ft -z -Xs -P
```

Options: -Ft = tar format; -z = gzip; -Xs = stream WAL while the backup runs; -P = progress display.

4.10 Verify physical backup files

```
kubectl -n "$NS" exec "$REPLICA" -- ls -lh /tmp/basebackup
```

```
kubectl -n "$NS" exec "$REPLICA" -- find /tmp/basebackup -maxdepth 1 -type f -printf "%f %s bytes\n"
```

Typical tar-format output includes base.tar.gz and WAL tar output when WAL streaming is included.

4.11 Copy the backup out to the workstation

```
mkdir -p backups/physical

PHYSICAL_DIR="backups/physical/basebackup-$(date +%Y%m%d-%H%M%S)"

kubectl -n "$NS" cp "$REPLICA:/tmp/basebackup" "$PHYSICAL_DIR"

find "$PHYSICAL_DIR" -maxdepth 2 -type f -ls | head -20
```

4.12 Remove password variable from the shell

```
unset REPL_PASS
```

4.13 Troubleshooting the exact errors seen in this lab

- could not translate host name "postgres-ha": the Service name is wrong or does not exist. Use kubectl get svc and set PRIMARY_SERVICE=acid-postgres-ha.
- no pg_hba.conf entry for replication connection ... user "postgres": the host is reachable, but postgres is not permitted for replication. Use the operator-managed replication/standby role and its Secret.
- no pg_hba.conf entry ... SSL encryption / no encryption: the server evaluated SSL and non-SSL HBA paths and neither matched that replication user/source. Do not bypass this with trust; fix role/HBA/authentication correctly.

5. Lab 3 - WAL Archiving Validation

5.1 Inspect archive settings

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_mode;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_command;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_library;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -x -c "SELECT * FROM pg_stat_archiver;"
```

5.2 Force a WAL switch and verify progress

```
BEFORE=$(kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT archived_count FROM pg_stat_archiver;")

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"

sleep 10

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT archived_count, failed_count, last_archived_wal, last_archive

echo "Archived before test: $BEFORE"
```

5.3 Check native Spilo backup tooling before changing archive_command

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'command -v wal-g || true; command -v wal-e || true; env | grep -E "^(USE_WALG|WAL
```

6. Lab 4 - Configure pgBackRest for Docker Kubernetes + GCS

Dry-run corrected. Only Lab 4 onward is replaced. Labs 0-3 remain exactly as in the supplied PDF.

Note: Use one pgBackRest identity everywhere: config /etc/pgbackrest/pgbackrest.conf and stanza postgres-ha. The earlier WAL timeout was caused by mixing this with /home/postgres/pgdata/pgbackrest.conf and stanza acid-postgres-ha.

6.1 Environment used by this lab

Namespace	postgres-ha
Cluster	acid-postgres-ha
Pods	acid-postgres-ha-0 / -1 / -2
Kubernetes	Docker Kubernetes (not GKE)
pgBackRest	2.59.1
GCS bucket	postgres-ha-backups-parameswari-2026
Stanza	postgres-ha
Config	/etc/pgbackrest/pgbackrest.conf
Cipher Secret	pgbackrest-repo-secret -> /etc/pgbackrest-secret/repo-cipher-pass
GCS key Secret	pgbackrest-gcs-key -> /etc/pgbackrest-gcs/gcs-key.json

6.2 Set variables and save the current CR

```
NS=postgres-ha; CLUSTER=acid-postgres-ha; STANZA=postgres-ha; PGBR_CONF=/etc/pgbackrest/pgbackrest.conf; BUCKET=postgres-ha-backups-parameswari-2026
```

```
kubectl -n "$NS" get pods -o wide
```

```
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml > "${CLUSTER}-before-pgbackrest.yaml"
```

6.3 Verify pgBackRest and preserve the existing cipher Secret

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do printf "%s: " "$pod"; kubectl -n "$NS" exec "$pod" -c postgres -- pgbackrest version; done
```

Expected: All three pods report pgBackRest 2.59.1.

Important: Do not recreate pgbackrest-repo-secret if it already exists. Replacing the passphrase can make encrypted backups unreadable.

```
kubectl -n "$NS" get secret pgbackrest-repo-secret
kubectl -n "$NS" get secret pgbackrest-repo-secret -o jsonpath="{.data.repo-cipher-pass}" >/dev/null && echo "repo-cipher-pass OK"
```

6.4 Configure GCS access for Docker Kubernetes

Docker Kubernetes cannot use repo1-gcs-key-type=auto because metadata.google.internal is unavailable. Use a mounted service-account JSON key.

```
PROJECT_ID=$(gcloud config get-value project); SA_NAME=pgbackrest-gcs; SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"; echo "$SA_EMAIL"
gcloud iam service-accounts describe "$SA_EMAIL" >/dev/null 2>&1 || gcloud iam service-accounts create "$SA_NAME" --display-name="pgBackRest GCS backup"
gcloud storage buckets add-iam-policy-binding "gs://$BUCKET" --member="serviceAccount:${SA_EMAIL}" --role="roles/storage.objectAdmin"
```

6.4A If key creation is blocked by Organization Policy

- Google Cloud Console -> IAM & Admin -> Organization Policies.
- Open the Active iam.disableServiceAccountKeyCreation constraint.
- At project level, override the parent and set enforcement to Off / Not enforced.
- Wait for propagation, then continue.

```
gcloud org-policies describe constraints/iam.disableServiceAccountKeyCreation --project="$PROJECT_ID"
test -f pgbackrest-gcs-key.json || gcloud iam service-accounts keys create pgbackrest-gcs-key.json --iam-account="$SA_EMAIL"
chmod 600 pgbackrest-gcs-key.json
kubectl -n "$NS" get secret pgbackrest-gcs-key >/dev/null 2>&1 || kubectl -n "$NS" create secret generic pgbackrest-gcs-key --from-file=gcs-key.json=./pgbackrest-gcs-key.json
kubectl -n "$NS" get secret pgbackrest-gcs-key -o jsonpath="{.data.gcs-key\.json}" >/dev/null && echo "gcs-key.json OK"
```

6.5 Discover the current primary and PGDATA

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"
```

```
PGDATA_ACTUAL=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "SHOW data_directory;"); echo "$PGDATA_ACTUAL"
```

Expected: The primary pod is printed. PGDATA should normally be /home/postgres/pgdata/pgroot/data.

6.6 Create pgbackrest.conf

Open the file with nano and paste the configuration block exactly as shown:

```
nano pgbackrest.conf
```

```
[postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=postgres-ha-backups-parameswari-2026
repo1-gcs-key-type=service
repo1-gcs-key=/etc/pgbackrest-gcs/gcs-key.json
repo1-path=/pgbackrest/postgres-ha
repo1-retention-full=2
repo1-retention-diff=4
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
archive-timeout=120
log-level-console=info

[global:archive-push]
process-max=2

[global:archive-get]
process-max=2
```

Note: If PGDATA_ACTUAL differs, change pg1-path before creating the ConfigMap.

```
kubectl -n "$NS" create configmap pgbackrest-config --from-file=pgbackrest.conf=./pgbackrest.conf --dry-run=client -o yaml | kubectl apply -f -
```

```
kubectl -n "$NS" get configmap pgbackrest-config -o jsonpath="{.data.pgbackrest\.conf}" | grep -E "repo1-gcs-bucket|repo1-gcs-key-type|repo1-gcs-key|repo1-path|archive-timeout"
```

6.7 Mount config, cipher Secret and GCS key

Create the following patch file. Keeping the JSON in a file makes the kubectl command short and easy to copy.

```
nano pgbackrest-additional-volumes-patch.json
```

```
{
  "spec": {
    "additionalVolumes": [
      {"name": "pgbackrest-config", "mountPath": "/etc/pgbackrest", "targetContainers": ["postgres"], "volumeSource": {"configMap": {"name": "pgbackrest-config"}}},
      {"name": "pgbackrest-secret", "mountPath": "/etc/pgbackrest-secret", "targetContainers": ["postgres"], "volumeSource": {"secret": {"secretName": "pgbackrest-repo-secret"}}},
      {"name": "pgbackrest-gcs-key", "mountPath": "/etc/pgbackrest-gcs", "targetContainers": ["postgres"], "volumeSource": {"secret": {"secretName": "pgbackrest-gcs-key"}}}
    ]
  }
}
```

```
python3 -m json.tool pgbackrest-additional-volumes-patch.json >/dev/null && echo "JSON OK"
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-additional-volumes-patch.json --dry-run=server -o yaml >/tmp/pgbackrest-volumes-preview.yaml
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-additional-volumes-patch.json
```

```
kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=300s
```

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do printf "%s: " "$pod"; kubectl -n "$NS" exec "$pod" -c postgres --sh -lc 'test -r /etc/pgbackrest/pgbackrest.conf && test -r /etc/pgbackrest-secret/repo-cipher-pass && test -r /etc/pgbackrest-gcs/gcs-key.json && echo OK || echo FAILED'; done
```

Expected: Every pod prints OK.

6.8 Create the pgBackRest stanza on the current primary

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass); pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha stanza-create'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass); pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Note: If stanza-create says unable to find primary cluster, the command was run on a replica. Re-detect PRIMARY and run stanza-create only on the pod where pg_is_in_recovery() = false.

6.9 Correct PostgreSQL archive_command

Note: This is the critical dry-run correction. PostgreSQL must archive with the SAME config and stanza used by the backup commands.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "show archive_mode; show archive_command;"
```

Create this small patch file:

```
nano pgbackrest-archive-command-patch.json
```

```
{
  "spec": {
    "postgresql": {
      "parameters": {
        "archive_mode": "on",
        "archive_command": "PGBACKREST_REPO1_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass) pgbackrest
--config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha archive-push %p"
      }
    }
  }
}
```

```
python3 -m json.tool pgbackrest-archive-command-patch.json >/dev/null && echo "JSON OK"
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-archive-command-patch.json --dry-run=server -o yaml > /tmp/archive-command-preview.yaml
```

```
kubectl -n "$NS" patch postgresql "$CLUSTER" --type merge --patch-file pgbackrest-archive-command-patch.json
```

```
kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=300s
```

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "show archive_command;"
```

Expected: archive_command contains /etc/pgbackrest/pgbackrest.conf, stanza postgres-ha and archive-push %p. It must not use /bin/true, /home/postgres/pgdata/pgbackrest.conf, or stanza acid-postgres-ha.

6.10 Run pgBackRest check and prove WAL archiving works

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha check'
```

```
BEFORE=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -Atqc "select archived_count from pg_stat_archiver;"); echo "$BEFORE"
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -c "select pg_switch_wal();"
```

```
sleep 10
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -x -c "select archived_count,failed_count,last_archived_wal,last_archived_time,last_failed_wal,last_failed_time from pg_stat_archiver;"
```

Expected: archived_count increases and there is no new archive failure. Only then continue to Lab 5.

7. Lab 5 - Full Backup with pgBackRest

Note: Do not start this lab until section 6.10 succeeds.

```
PRIMARY=$(for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do v=$(kubectl -n "$NS" exec "$pod" -c postgres -- psql -U postgres -Atqc "select pg_is_in_recovery();"); if [ "$v" = "f" ]; then echo "$pod"; break; fi; done); echo "$PRIMARY"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=full backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: The newest backup label ends with F and status is OK.

7.1 Create test data after the full backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "create table if not exists dr_lab_events(id bigint generated always as identity primary key,event_time timestamptz default clock_timestamp(),note text); insert into dr_lab_events(note) values ('after full backup'); select * from dr_lab_events order by id;"
```

8. Lab 6 - Incremental Backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "insert into dr_lab_events(note) values ('before incremental backup');"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=incr backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: The newest backup label ends with I.

9. Lab 7 - Differential Backup

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- psql -U postgres -d banking -c "insert into dr_lab_events(note) values ('before differential backup');"

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --type=diff backup'

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
```

Expected: The newest backup label ends with D.

10. Repository, Compression, Encryption and Retention

- Repository: gs://postgres-ha-backups-parameswari-2026/pgbackrest/postgres-ha
- Compression: zst level 3
- Encryption: AES-256-CBC
- Retention: 2 full / 4 differential
- Archive timeout: 120 seconds

Note: Keep the original cipher passphrase. Losing or replacing it makes encrypted backups unrecoverable.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha info'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --dry-run expire'
```

11. Lab 8 - Restore / Full Cluster Recovery

Safety: Never restore into active acid-postgres-ha PVCs. Use isolated recovery storage.

Use an isolated PVC/toolbox that mounts all three pgBackRest resources: pgbackrest-config, pgbackrest-repo-secret and pgbackrest-gcs-key. The restore command must continue to use config /etc/pgbackrest/pgbackrest.conf and stanza postgres-ha.

```
kubectl -n "$NS" exec pgbackrest-restore-toolbox -c postgres -- sh -lc 'mkdir -p /restore/pgdata; export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --pg1-path=/restore/pgdata restore'
```

Note: To restore the latest eligible backup, omit --set. Use --set only with a real backup label.

12. Lab 9 - Point-in-Time Recovery (PITR)

PITR also uses isolated storage and the same config, stanza, cipher Secret and GCS key.

```
kubectl -n "$NS" exec pgbackrest-pitr-toolbox -c postgres -- env TARGET_TIME="$TARGET_TIME" sh -lc 'mkdir -p /restore/pgdata; export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --config=/etc/pgbackrest/pgbackrest.conf --stanza=postgres-ha --pg1-path=/restore/pgdata --type=time --target="$TARGET_TIME" --target-action=promote restore'
```

Expected: Rows committed before TARGET_TIME are present and later unwanted changes are absent.

13. Patroni / Failover Rules

- Rediscover PRIMARY before stanza-create, check and every backup.
- acid-postgres-ha Service follows the current primary; direct pod names do not.
- After failover, inspect pgBackRest info before deciding whether another backup is needed.
- Never edit active PGDATA manually.

14. Troubleshooting Appendix - errors already seen

Error / symptom	Correction
repo1-cipher-pass must have a value	Export PGBACKREST_REPO1_CIPHER_PASS inside the same kubectl exec command.
metadata.google.internal not found	Use GCS key-type=service with the mounted JSON key.
missing stanza path	Run stanza-create on the current primary.
unable to find primary cluster	You ran stanza-create on a replica; rediscover PRIMARY.
archive_command '/bin/true' must contain pgbackrest	Apply section 6.9 and verify archive_command.
WAL segment was not archived before timeout	Verify archive_command uses /etc/pgbackrest/pgbackrest.conf + stanza postgres-ha, then run pgBackRest check before retrying backup.
old /home/postgres/pgdata/pgbackrest.conf or stanza acid-postgres-ha	Old archive identity is still active. Apply section 6.9.
cat/etc/... not found	Use cat /etc/... with a space.
invalid JSON / newline in string literal	Validate patch files first with python3 -m json.tool.

15. RPO / RTO and Suggested Backup Schedule

- WAL archiving: continuous.
- Full backup: weekly lab baseline; production cadence depends on RPO/RTO and data volume.
- Differential: useful between full backups for a shorter restore chain.
- Incremental: smaller backup windows but a longer dependency chain.
- Restore/PITR drills: run regularly on isolated storage.

16. Final Coverage Checklist

- ☐ Primary discovered dynamically
- ☐ GCS service account/key configured

- ☐ Original cipher Secret preserved
- ☐ pgBackRest config mounted on all pods
- ☐ GCS key mounted on all pods
- ☐ Stanza created on current primary
- ☐ archive_command uses /etc/pgbackrest/pgbackrest.conf
- ☐ archive_command uses stanza postgres-ha
- ☐ pgBackRest check succeeds
- ☐ WAL archiving succeeds before full backup
- ☐ Full backup succeeds
- ☐ Incremental backup succeeds
- ☐ Differential backup succeeds
- ☐ Isolated restore/PITR use same config, stanza and credentials
- ☐ Active Patroni PVCs untouched